

DPDP COMPLIANCE AUDIT

SaaS-Project

Digital Personal Data Protection Act 2023

64/100

Grade C

Needs Work

Scanned: 25 March 2026

Files indexed: 16

Languages: FastAPI, SQLAlchemy, Pydantic, JWT, Python, React

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

The application collects usernames and hashed passwords for authentication. The biggest DPDP risk is the potential for unauthorized access if JWT secrets are compromised or if password hashing is insufficient, although the code uses SHA-256 with a salt.

Validation Summary Rule findings: 12 confirmed, 2 false positives suppressed • AI gaps: 3 dual-validated • Score: 65 (rules) → 64 (integrated)

Total findings: 15

HIGH: 4

MEDIUM: 4

INFO: 4

PASS: 1

Tech Stack: FastAPI, SQLAlchemy, Pydantic, JWT, Python

Risk Profile: The application collects usernames and hashed passwords for authentication. The biggest DPDP risk is the potential for unauthorized access if JWT secrets are compromised or if password hashing is insufficient, although the code uses SHA-256 with a salt.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	11/22 (50%)	WARN
Section 6(6)	8	0/8 (0%)	FAIL
Section 8(1)	18	18/18 (100%)	PASS
Section 8	10	10/10 (100%)	PASS
Section 8(3)	8	8/8 (100%)	PASS
Section 8(4)	8	0/8 (0%)	FAIL
Section 8(6)	8	6.8/8 (85%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	2.1/6 (35%)	FAIL
Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 2.2–7.8 engineering days

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Application Logs

PII fields: password

main.py

Rule	Section	Severity	File
------	---------	----------	------

CONSENT_WITHDRAWAL_MISSING	Section 6(6) — Consent Withdrawal	HIGH	N/A
AUDIT_TRAIL_MISSING	Section 8(4) — Audit Trail	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
DATA_ACCESS_MISSING	Section 11 — Right to Access	MEDIUM	N/A
NO_BREACH_ALERTING	Section 8(6) — Breach	LOW	N/A
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	backend/main.py
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
CONSENT_PRESENT	Section 6 — Consent	PASS	backend/auth.py
CONSENT_MISSING	Section 6 — Consent	HIGH	MULTIPLE
	Section 9 — Children	MEDIUM	
	Section 5 — Purpose Limitation	MEDIUM	
	Section 4 & 6 — Consent for Third-Party Sharing	MEDIUM	

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (9–33 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Withdrawal Missing (1–4 hrs)
- Audit Trail Missing (4–12 hrs)
- Grievance Officer Missing (1–4 hrs)
- Consent Missing — MULTIPLE (3–13 hrs)

Week 2 — High Priority (8–30 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Data Access Missing (2–6 hrs)
- (2–8 hrs)
- (2–8 hrs)
- (2–8 hrs)

Week 4 — Advisory (4–14 hours)

Address before next compliance review.

- No Breach Alerting (2–6 hrs)
- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 21–77 hours (2.6–9.6 engineering days)

Details

Finding 1 of 15

[Section 6(6) — Consent Withdrawal]

1. [HIGH] CONSENT_WITHDRAWAL_MISSING — N / A

Estimated fix time: 1–4 hours

Consent is collected but no withdrawal mechanism detected. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent. Users must be able to revoke consent at any time.

Risk:

The absence of a consent withdrawal mechanism means users cannot easily revoke their consent for data processing, violating DPDP Section 6(6). This can lead to continued processing of personal data against an individual's wishes, potentially resulting in user dissatisfaction and legal penalties.

Fix:

1. Create a new API endpoint (e.g., `/consent/withdraw`) that allows authenticated users to revoke their consent.
2. Implement a mechanism in the database to track user consent status (e.g., a boolean flag or a timestamp for consent withdrawal).
3. Update relevant services to stop processing personal data for users who have withdrawn consent.
4. Ensure the withdrawal process is as easy as the consent process, potentially by providing a clear link or button in the user's profile or settings.

DPDP:

Section 6(6) — Consent Withdrawal

Example:

```
from fastapi import APIRouter, Depends, HTTPException, status from sqlalchemy.orm import Session from backend.auth import get_current_user # Assuming you have a way to get the current user from backend.database import get_db # Assuming you have a get_db function router = APIRouter() @router.post('/consent/withdraw') def withdraw_consent(current_user: auth.UserDB = Depends(get_current_user), db: Session = Depends(get_db)): # Logic to mark user consent as withdrawn in the database # e.g., current_user.consent_given = False # db.commit() return {"message": "Consent withdrawn successfully"}
```

Finding 2 of 15

[Section 8(4) — Audit Trail]

2. [HIGH] AUDIT_TRAIL_MISSING — N / A:108

Evidence snippet: The signup endpoint is defined here. There is no logging or tracking of user creation or authentication events.

Estimated fix time: 4–12 hours

No audit trail infrastructure detected. DPDP Section 8(4) requires complete records of personal data processing activities. No access logging, audit model, or change tracking found in the codebase.

Risk:

The absence of an audit trail means there are no records of personal data processing activities, violating DPDP Section 8(4). This lack of accountability makes it difficult to demonstrate compliance, investigate data breaches, or understand data flows, increasing the risk of regulatory non-compliance.

Fix:

1. Create an `AuditLog` model in `backend/models.py` to store event details (timestamp, user, action, data).
2. Implement a function to add audit log entries, potentially using a decorator or middleware.
3. Apply this logging mechanism to critical operations like user signup, login, data modification, and consent changes.
4. Ensure audit logs are stored securely and retained according to policy.

DPDP:

Section 8(4) — Audit Trail

Example:

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey from sqlalchemy.sql
import func class AuditLog(Base): __tablename__ = "audit_logs" id = Column(Integer,
primary_key=True, index=True) timestamp = Column(DateTime(timezone=True),
server_default=func.now()) user_id = Column(Integer, ForeignKey("users.id")) action =
Column(String) details = Column(String, nullable=True)
```

Finding 3 of 15

[Section 13 — Grievance Redressal]

3. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A:13

Evidence snippet: The FastAPI app is initialized here, but no specific route or configuration for grievance redressal is present.

Estimated fix time: 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route ****and**** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

Failure to publish contact details for a Grievance Officer or DPO prevents individuals from lodging complaints or seeking redressal, violating DPDP Section 13. This can lead to unresolved grievances, damage to reputation, and potential legal challenges.

Fix:

1. Create a new API endpoint (e.g., `/grievance/contact`) that returns the contact details of the Grievance Officer or DPO.
2. Alternatively, add a static page or a dedicated section in the application's documentation that clearly lists the grievance contact information.
3. Ensure the contact information provided is accurate and monitored.
4. Publish this information prominently within the application or on its associated website.

DPDP:

Section 13 — Grievance Redressal

Example:

```
from fastapi import APIRouter
router = APIRouter()
@router.get("/grievance/contact")
def get_grievance_contact():
    return {
        "message": "For any grievances, please contact our Data Protection Officer.",
        "email": "dpo@example.com", # Replace with actual email
        "phone": "+1234567890" # Optional: Replace with actual phone
    }
```

Finding 4 of 15

[Section 11 — Right to Access]

4. [MEDIUM] DATA_ACCESS_MISSING — N / A:27**Evidence snippet:** The application is a FastAPI application, which is a web framework that can be used to build APIs.**Estimated fix time:** 2–6 hours

No endpoint detected that allows authenticated users to retrieve their own personal data. Under DPDP Section 11, Data Principals have the right to access information about their personal data being processed.

Risk:

The absence of a data access endpoint means users cannot exercise their right to access their personal data, which is a violation of DPDP Section 11. This can lead to user distrust and potential regulatory penalties.

Fix:

1. Create a new GET endpoint in `backend/main.py` (e.g., `/users/me`) that requires authentication.
2. Inside the endpoint, retrieve the currently authenticated user's username from the token.
3. Query the database for the user's record using their username.
4. Return the user's personal data in a JSON response, excluding sensitive fields like hashed passwords.

DPDP:

Section 11 — Right to Access: Data Principals have the right to obtain confirmation that their personal data is being processed and to access that data.

Example:

```
from fastapi import Depends from sqlalchemy.orm import Session from backend.auth import get_current_user # Assuming you have a function to get current user @app.get('/users/me') def read_users_me(current_user: models.UserDB = Depends(get_current_user)): # Return user data, excluding sensitive info like password hash return { "username": current_user.username, "email": current_user.email # Assuming email is a PII field }
```

Finding 5 of 15

[Section 8(6) — Breach]

5. [LOW] NO_BREACH_ALERTING — N / A:27**Evidence snippet:** The application is a FastAPI application, which is a web framework that can be used to build APIs.**Estimated fix time:** 2–6 hours

Structured logging detected but no breach notification mechanism found. DPDP Section 8(6) requires notifying affected individuals and the Authority without undue delay when a breach occurs. Logging alone is insufficient — implement automated alerting (Sentry, PagerDuty, or equivalent) with escalation to a compliance team.

Risk:

The lack of a breach notification mechanism means individuals affected by a data breach will not be promptly informed, hindering their ability to protect themselves and violating DPDP Section 8(6). This can lead to reputational damage and regulatory fines.

Fix:

1. Integrate a third-party error tracking and alerting service (e.g., Sentry, PagerDuty) into the application.
2. Configure the service to monitor for critical errors and exceptions that could indicate a data breach.
3. Set up automated notifications to a designated compliance team or security contact upon detection of a potential breach.
4. Ensure logs are detailed enough to aid in post-breach analysis and reporting.

DPDP:

Section 8(6) — Breach: Data Fiduciaries must notify the affected Data Principal and the Authority without undue delay when a personal data breach occurs.

Example:

```
import sentry_sdk
sentry_sdk.init( dsn="YOUR_SENTRY_DSN", # Set traces_sample_rate to 1.0 to
capture 100% of transactions for performance monitoring. # We recommend adjusting this value
in production. traces_sample_rate=1.0, # If you wish to associate users to events, uncomment
the following line # and install the appropriate integration:
https://docs.sentry.io/product/integrations/platform-integration/ # with
sentry_sdk.push_scope() as scope: # scope.user = {"username": "john.doe"} ) # Example of how
to capture an exception: try: # ... code that might raise an exception ... pass except
Exception as e: sentry_sdk.capture_exception(e)
```


Finding 6 of 15

[Section 11 — Data Portability]

6. [LOW] DATA_PORTABILITY_MISSING — N / A:27**Evidence snippet:** The application is a FastAPI application, which is a web framework that can be used to build APIs.**Estimated fix time:** 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The absence of a data export or portability endpoint prevents users from obtaining their personal data in a machine-readable format, violating DPDP Section 11. This can lead to non-compliance and user dissatisfaction.

Fix:

1. Create a new GET endpoint in `backend/main.py` (e.g., `/users/me/export`) that requires authentication.
2. Inside the endpoint, retrieve the currently authenticated user's data, similar to the data access endpoint.
3. Format the retrieved data into a commonly used, machine-readable format such as JSON or CSV.
4. Return the formatted data with appropriate headers for file download (e.g., `Content-Disposition: attachment; filename="user\_data.json"`).

DPDP:

Section 11 — Data Portability: Data Principals have the right to receive their personal data in a structured, commonly used, and machine-readable format.

Example:

```
from fastapi import Response, import json @app.get('/users/me/export') def
export_users_me(current_user: models.UserDB = Depends(get_current_user)): user_data = {
"username": current_user.username, "email": current_user.email # Assuming email is a PII field
# Add other relevant PII fields here } json_data = json.dumps(user_data, indent=2) return
Response(content=json_data, media_type='application/json', headers={"Content-Disposition":
"attachment; filename=\"user_data.json\""})
```

Finding 7 of 15

[Section 7 — Legitimate Use]

7. [INFO] INTERNAL_ROUTE_PII_ACCESS — backend / main.py

Internal file (main.py) accesses PII. Verify access is restricted to authorised contexts and PII is not exposed to logs or error responses.

Finding 8 of 15

[Section 5 — Notice]

8. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 9 of 15

[Section 5 — Purpose Limitation]

9. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 10 of 15

[Section 16 — Cross-Border Transfer]

10. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 11 of 15

[Section 6 — Consent]

11. [PASS] CONSENT_PRESENT — backend / auth.py

Framework auth/consent middleware detected in this file.

Finding 12 of 15

[Section 6 — Consent]

12. [HIGH] CONSENT_MISSING — MULTIPLE:6

Evidence snippet: `const [username, setUsername] = useState(""); // username, not email`

Estimated fix time: 3–13 hours

Affected files (3):

- frontend/src/Pages/Signup.jsx
- frontend/src/Pages/Dashboard.jsx
- frontend/src/Pages/Login.jsx

Personal data collection and API endpoint detected with no consent mechanism. Detected in 3 files: Signup.jsx, Dashboard.jsx, Login.jsx

Risk:

The absence of a consent mechanism means personal data may be processed without explicit user permission, violating DPDP Section 6(1). This can lead to non-compliance, user distrust, and penalties for unauthorized data processing.

Fix:

1. Modify the signup endpoint to include a checkbox or clear affirmative action for users to consent to terms and privacy policy.
2. Store a record of consent, including the timestamp and version of the policy agreed upon, linked to the user's account.
3. Implement a clear and accessible privacy policy that details data processing activities.
4. Ensure that data processing activities not covered by explicit consent are either removed or require separate consent.

DPDP:

Section 6(1) — Consent

Example:

```
from fastapi import APIRouter, Depends, HTTPException, status from pydantic import BaseModel
from sqlalchemy.orm import Session from backend.database import get_db class
ConsentRequest(BaseModel): username: str password: str consent_given: bool = False
@router.post('/signup') def signup(user_data: ConsentRequest, db: Session = Depends(get_db)):
if not user_data.consent_given: raise HTTPException(status_code=400, detail="Consent is
required to create an account.") # ... rest of signup logic ...
```

Finding 13 of 15

[Section 9 — Children]

13. [MEDIUM] — CODEBASE-WIDE

Missing age verification for signup

Finding 14 of 15

[Section 5 — Purpose Limitation]

14. [MEDIUM] — CODEBASE-WIDE

PII (username) used in analytics without clear purpose

Finding 15 of 15

[Section 4 & 6 — Consent for Third-Party Sharing]

15. [MEDIUM] — CODEBASE-WIDE

Missing consent for third-party data sharing (Groq API)

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] Missing age verification for signup

Confidence: 67%

Section 9 — Children

The ``frontend/src/Pages/Signup.jsx`` component handles user registration, but there is no mention of an age input field or any age verification logic. The ``UserDB`` model in ``backend/auth.py`` also lacks fields for age or date of birth, suggesting no mechanism to prevent or identify children's data collection.

Recommendation: Implement an age verification step during the user signup process in ``frontend/src/Pages/Signup.jsx``. Add corresponding backend validation and data storage (e.g., a ``date_of_birth`` field in ``backend/auth.py``'s ``UserDB`` model) to ensure compliance with Section 9 regarding children's data.

[AI-INFERRED] PII (username) used in analytics without clear purpose

Confidence: 70%

Section 5 — Purpose Limitation

The ``data_flows`` explicitly states: 'Username collected via GET /analytics used for user identification in analytics retrieval'. While analytics is a general purpose, the context does not provide details on the specific necessity, scope, or explicit consent for using PII (username) in this manner, especially given the general ``CONSENT_MISSING`` finding.

Recommendation: Review the ``GET /analytics`` endpoint in ``backend/main.py`` to ensure that the collection and use of usernames for analytics purposes are strictly necessary, clearly defined, and align with the principle of purpose limitation under Section 5. Consider anonymizing or pseudonymizing data where possible to reduce PII exposure.

[AI-INFERRED] Missing consent for third-party data sharing (Groq API)

Confidence: 86%

Section 4 & 6 — Consent for Third-Party Sharing

The application uses 'Groq API (backend/rag.py:23)' as a third-party service, and 'Query text collected via POST /query sent to RAG pipeline'. The rule engine explicitly noted `third_party_checked: false`, indicating no check was performed. Sharing user query data (which may contain PII) with a third-party API without explicit consent is a significant DPDP compliance risk.

Recommendation: Implement a clear consent mechanism (e.g., a checkbox during signup or before the first query) in `frontend/src/Pages/Signup.jsx` or `frontend/src/Pages/Dashboard.jsx` to obtain explicit consent for sharing user query data with the Groq API. Ensure this consent is recorded and respected in `backend/rag.py` before making API calls.

Deep Compliance Review

Overall Risk: CRITICAL

The application presents critical DPDP compliance risks due to systemic architectural flaws in personal data handling and authentication security. Uncontrolled ingestion and retention of PII across query, source, and feedback models violate data minimization and purpose limitation principles. Furthermore, weak password hashing, default salts, and insecure JWT secret management expose user credentials and session integrity, creating a high risk of unauthorized access and data breaches. These issues collectively undermine core data protection principles, leading to significant non-compliance exposure.

Estimated remediation: 14 engineering day(s)

Most Critical Action: Upgrade password hashing to an industry-standard algorithm (e.g., bcrypt or Argon2) with unique, cryptographically strong salts for each user, and ensure the JWT secret key is securely managed via environment variables.

Layer Breakdown

Configuration & Secrets — 0 finding(s). No compliance issues identified in this layer. The provided files are configuration for development tools (ESLint and Tailwind CSS) and do not directly handle personal data, implement data protection controls, or manage data flows relevant to DPDP compliance.

API Routes & Controllers — 3 finding(s). This layer exhibits critical vulnerabilities in authentication mechanisms, including weak password hashing, a default salt, and an insecure JWT secret key, which directly undermine data security controls. Additionally, there's a compliance risk related to the potential flow of PII into the feedback system without explicit consent or purpose limitation.

Authentication & Session — 2 finding(s). The authentication and session management layer exhibits critical security vulnerabilities related to password protection. The use of a weak hashing algorithm and a default, easily guessable salt directly compromises the security of user credentials, which are considered personal data under DPDP. Addressing these issues is paramount for compliance with data security principles.

Data Models & Storage — 3 finding(s). This layer's data models exhibit significant architectural gaps regarding the handling of Personal Information (PII). Generic string and dictionary fields across query, source, and feedback models allow for uncontrolled ingestion, processing, and retention of PII without explicit mechanisms for data minimization, purpose limitation, or security. This design creates a high compliance risk for DPDP principles.